

# The Library, Line by Line

`fracmem_final.py` — Every Function Explained, In Order

July 19, 2026

**What this document is.** A complete walkthrough of the actual, working code for the compressed fractional-order memory filter – the same code that produced every number in the companion theory document. Every function is shown in full and explained: what it computes, why it is written the way it is, and how it connects to the mathematics. Read alongside `final_theory.pdf` (which proves *why* the method works) this document explains *how it is implemented*. The file is 255 lines, organized into five parts that mirror the five parts of the theory document exactly.

## Contents

|            |                                                            |          |
|------------|------------------------------------------------------------|----------|
| <b>1</b>   | <b>How the file is organized</b>                           | <b>1</b> |
| <b>I</b>   | <b>GL Weights: The Exact Target Kernel</b>                 | <b>2</b> |
| <b>II</b>  | <b>SOE Quadrature: Decay Rates and Initial Weights</b>     | <b>3</b> |
| <b>III</b> | <b>The Recursion: What Actually Runs at Inference Time</b> | <b>5</b> |
| <b>IV</b>  | <b>Cross-Validated Ridge Regression</b>                    | <b>6</b> |
| <b>V</b>   | <b>The Assembled Pipeline</b>                              | <b>7</b> |
| <b>2</b>   | <b>Putting it together: a complete usage example</b>       | <b>9</b> |
| <b>3</b>   | <b>Verified results, from this exact code</b>              | <b>9</b> |

## 1 How the file is organized

`fracmem_final.py` has five parts, in dependency order – each part only uses things already defined above it:

1. **GL weights** – the exact target kernel (pure math, no approximation).
2. **SOE quadrature** – derives the decay rates and an initial set of weights, analytically, from the kernel in Part 1. No signal data is touched anywhere in this part.

3. **Deployment-time recursion and feature extraction** – the actual recursive filter that runs at inference time, plus a fast batched version used only while fitting.
4. **Cross-validated ridge regression** – fits the linear weights to training data, honestly.
5. **The assembled pipeline** – a single class wrapping Parts 1–4 into `.fit(...)` and `.predict(...)`.

Only Part 4 (and the `.fit` method in Part 5, which calls it) ever looks at training data. Parts 1–3 are pure, deterministic mathematics.

## Part I

# GL Weights: The Exact Target Kernel

```

1 def gl_weights(alpha: float, n: int) -> np.ndarray:
2     """Exact GL weights  $w_0..w_{n-1}$  via the stable recursion
3          $w_0 = 1, w_j = w_{j-1} * (1 - (1+\alpha)/j)$ .
4     """
5     w = np.empty(n, dtype=np.float64)
6     w[0] = 1.0
7     for j in range(1, n):
8         w[j] = w[j - 1] * (1.0 - (1.0 + alpha) / j)
9     return w

```

This is equation (3) of the theory document, implemented directly: allocate an array of length  $n$ , set the first entry to 1 (that is  $w_0$ ), then walk forward multiplying each entry by  $(1 - \frac{1+\alpha}{j})$  to get the next one. There is deliberately no direct call to a Gamma-function library here – the recursion is used specifically *because* it avoids the large cancellations that computing  $\binom{\alpha}{j}$  directly from Gamma functions would suffer from at large  $j$ . This function never looks at any signal; given  $\alpha$  and how many weights you want, it returns the same numbers every time.

```

1 def full_gl_derivative(x: np.ndarray, alpha: float, h: float, w: np.ndarray) -> np.
   ndarray:
2     """The gold-standard, exact (but  $O(n)$ -memory) fractional derivative,
3     via direct convolution against the full weight sequence."""
4     n = len(x)
5     y = np.convolve(x, w[:n])[:n]
6     return y * (h ** (-alpha))

```

This is the direct, uncompressed implementation of equation (2): convolve the signal  $x$  against the weights  $w$ , keep only the first  $n$  samples of the convolution (`np.convolve` in ‘full’ mode returns a longer array than either input; the causal, “what we actually want” part is the first  $n$  entries), then multiply by  $h^{-\alpha}$  to convert from the raw weighted sum into physical derivative units. **This function is never used for deployment** – it needs  $O(n)$  weights and  $O(n)$  work every time it is called on a length- $n$  signal, exactly the problem this whole project exists to avoid. It exists purely as the “ground truth” the compressed filter is checked against.

```

1 def local_exact_term(x: np.ndarray, alpha: float, h: float, L: int, w: np.ndarray) ->
   np.ndarray:
2     """The near/local sum  $j=0..L-1$ , computed exactly -- an ordinary length- $L$ 
3     FIR filter, never approximated."""
4     y = np.convolve(x, w[:L])[:len(x)]
5     return y * (h ** (-alpha))

```

Identical idea, but convolving against only the *first*  $L$  weights (`w[:L]`) instead of the full sequence. This is the “local” half of the local-plus-tail split: an ordinary, short, exact FIR filter of fixed length  $L$  (typically 32), costing  $L$  multiply-adds per sample regardless of how long the signal runs. **This term is never approximated anywhere in the method** – it is always computed exactly, both while fitting and while deployed.

## Part II

# SOE Quadrature: Decay Rates and Initial Weights

```

1 def soe_nodes_and_weights(alpha: float, L: int, m_max: int, p: int):
2     """Log-spaced quadrature discretization of the Laplace representation
3      $j^{-\beta} = 1/\Gamma(\beta) * \int_0^\infty s^{\beta-1} e^{-sj} ds$ ,  $\beta=\alpha+1$ 
4     Returns (lambda, c) with  $\lambda_i = \exp(-s_i)$ ,  $p$  nodes covering the
5     integrand's significant range, log-uniformly spaced."""
6     beta = alpha + 1.0
7     A = 1.0 / abs(gamma(-alpha))
8
9     s_min = 1.0 / max(m_max, 1)
10    s_max = 8.0
11    u_min, u_max = np.log(s_min), np.log(s_max)
12    du = (u_max - u_min) / p
13    u = u_min + (np.arange(p) + 0.5) * du
14    s = np.exp(u)
15
16    lam = np.exp(-s)
17    c = (A / gamma(beta)) * du * (s ** beta) * np.exp(-s * L)
18    return lam, c

```

Line by line: `beta = alpha + 1.0` is  $\beta$  from Theorem 5.1 of the theory document (the exponent of the power law being represented,  $j^{-\beta}$ ). `A` is the asymptotic prefactor  $1/|\Gamma(-\alpha)|$  from the weight asymptotic. `s_min` and `s_max` set the range of decay rates the quadrature needs to cover: `s_min = 1/m_max` is small enough to represent decay all the way out to the furthest lag of interest ( $m_{\max}$  samples past  $L$ ); `s_max = 8` is fixed because  $e^{-8} \approx 3.4 \times 10^{-4}$  per sample is already far below any meaningful signal-to-noise floor – nodes decaying faster than that would add cost without adding accuracy.

Taking `ln` of `s_min` and `s_max` switches to *log*-space, and `u` places  $p$  points evenly spaced in that log-space (the `+0.5` offset centers each point in its own sub-interval, the standard midpoint-rule placement). `s = np.exp(u)` converts back: the actual  $s_i$  values are *log-uniformly* spaced, dense near the small- $s$  (slow-decay) end and sparse near the large- $s$  (fast-decay) end, exactly matching where the integrand  $s^{\beta-1}e^{-sj}$  actually carries weight across many orders of magnitude of  $j$ .

Finally `lam = np.exp(-s)` gives the decay rates  $\lambda_i = e^{-s_i}$  directly, and `c` is the rectangle-rule quadrature weight for node  $i$ : the integrand  $s^{\beta-1}e^{-sj}$  evaluated with  $j = L$  (since the tail starts there), times the interval width `du`, times the constants. **No signal, no training data, nothing but  $\alpha$ ,  $L$ , and  $p$  go into this function.**

```

1 def soe_refit_weights(alpha: float, L: int, w: np.ndarray, lam: np.ndarray, j_max: int
2     ,
3     n_samples: int = 400, reg: float = 0.1) -> np.ndarray:

```

```

3  """Linear least-squares refit of the weights against the EXACT GL tail
4  weights (never against any signal) -- corrects the small mismatch
5  between the asymptotic power law and the true kernel near j=L."""
6  m_grid = np.unique(np.round(np.exp(np.linspace(0, np.log(j_max - L), n_samples))).
7                      astype(int))
8  m_grid = m_grid[m_grid <= (j_max - L)]
9  j_grid = L + m_grid
10 target = w[j_grid]
11
12 M = lam[None, :] ** m_grid[:, None]
13 wts = 1.0 / np.maximum(np.abs(target), 1e-300) # fit RELATIVE error uniformly
14 Mw = M * wts[:, None]
15 tw = target * wts
16
17 p = M.shape[1]
18 Mw_reg = np.vstack([Mw, np.sqrt(reg) * np.eye(p)])
19 tw_reg = np.concatenate([tw, np.zeros(p)])
20 c, *_ = np.linalg.lstsq(Mw_reg, tw_reg, rcond=None)
    return c

```

The quadrature weights above approximate the *asymptotic* power law; this function refits  $c$  (holding  $\lambda$  fixed) against the *exact* weight sequence  $w$  from Part I, to remove that small asymptotic-vs-exact mismatch. `m_grid` builds a log-spaced sample of lag offsets from 0 to  $j_{\max} - L$  (again log-spacing, for the same reason as above – the tail spans orders of magnitude); `target = w[j_grid]` reads off the true weight at each sampled lag. `M = lam[None, :] * m_grid[:, None]` builds the design matrix (each column is one exponential basis function  $\lambda_i^m$  evaluated at every sampled lag  $m$ ) – this is literally  $F^\top$  from Theorem 9.2 of the theory document, with  $j_{\max} - L$  (well, the sampled subset of it) playing the role of the training points. The rest is a *weighted, ridge-regularized* version of the same normal-equations idea: `wts = 1/|target|` makes the fit match *relative*, not absolute, error (needed because the weights span orders of magnitude and an unweighted fit would be dominated by the first few, largest entries); a small ridge penalty (`reg=0.1`, deliberately larger than any regularization strength used later in Part IV, since this is fitting a smooth, exactly-known target rather than noisy signal data) keeps the solve numerically well-behaved; `np.linalg.lstsq` solves the resulting (regularized, weighted) least-squares problem in one call.

**Why this still counts as “no data.”** `soe_refit_weights` does perform a regression – but its target,  $w$ , is the exact GL kernel from Part I, a deterministic mathematical object computable to arbitrary precision from  $\alpha$  alone. No actual signal, and no randomness, ever enters this computation. This is a genuinely different thing from Part IV’s regression, whose target is the empirical behavior of real training signals.

```

1  def soe_tail_kernel(alpha: float, L: int, p: int, w: np.ndarray, j_max: int):
2      """Full classical construction: nodes from quadrature, weights refit to
3      the exact kernel. (lambda, c) here are the PURE, zero-data, provably
4      grounded classical filter -- Part III of the theory document."""
5      m_max = j_max - L
6      lam, c0 = soe_nodes_and_weights(alpha, L, m_max, p)
7      c = soe_refit_weights(alpha, L, w, lam, j_max)
8      return lam, c

```

A thin wrapper chaining the two functions above: derive nodes and an initial weight guess, then refit the weights against the exact kernel, and return the final  $(\lambda, c)$  pair. This pair, on its own, is the certified classical filter – everything from here on is about improving  $c$  (never  $\lambda$ ) using

data.

## Part III

# The Recursion: What Actually Runs at Inference Time

```
1 def diagonal_recurrence(x_delayed: np.ndarray, lam: np.ndarray, c: np.ndarray) -> np.
  ndarray:
2     """THE deployed filter: p independent one-line recursions.
3         m_i[k] = lam_i * m_i[k-1] + x_delayed[k]
4         out[k] = c . m[k]
5         O(p) multiply-adds per sample, O(p) persistent memory -- constant,
6         regardless of how long the signal runs. x_delayed is x shifted by L
7         samples (see 'delay' below)."""
8     n = len(x_delayed)
9     p = len(lam)
10    m = np.zeros(p)
11    out = np.empty(n)
12    for k in range(n):
13        m = lam * m + x_delayed[k]
14        out[k] = c @ m
15    return out
```

This is the entire deployed filter, and it is the most important function in the file to understand precisely because it is the one that actually has to run, forever, on the target hardware. `m = np.zeros(p)` is the persistent state:  $p$  numbers, one per mode, initialized to zero. The loop runs once per input sample; each iteration does exactly one vector multiply-add (`lam * m + x_delayed[k]`, updating every mode's state using only its own previous value and the current input – this is equation (4) of the theory document, run elementwise for all  $p$  modes at once) followed by one dot product (`c @ m`) to produce the output. **Nothing here grows with  $k$ :** the array `m` is always exactly  $p$  numbers long, and each loop iteration does exactly the same, fixed amount of work, no matter whether  $k$  is 10 or 10,000,000. This is the concrete, working answer to the “unbounded memory growth” problem posed at the very start of the theory document.

```
1 def delay(x: np.ndarray, L: int) -> np.ndarray:
2     """x shifted right by L samples, zero-padded -- the tail's own input
3         only starts once the exact local window has consumed its L samples."""
4     out = np.zeros_like(x)
5     if x.ndim == 1:
6         if len(x) > L:
7             out[L:] = x[:len(x) - L]
8     else:
9         n = x.shape[-1]
10        if n > L:
11            out[..., L:] = x[..., :n - L]
12    return out
```

A small utility: shifts a signal right by  $L$  samples (with zeros filling the first  $L$  entries). The tail recursion above is only ever fed *this* shifted signal, not the raw one – because the tail's job is to explain everything from lag  $L$  onward, and feeding it the raw, unshifted signal would double-count the first  $L$  samples that the exact local window (`local_exact_term`, Part I) already

accounts for. The function handles both a single signal (1-D array) and a batch of several signals stacked into a 2-D array (used only during fitting, below) via the same code path.

```

1 def mode_features(lam: np.ndarray, x_delayed: np.ndarray) -> np.ndarray:
2     """Fast batched computation of every mode's feature trace, for FITTING
3     (not deployment) -- (p, B, T) array, one row per mode, via lfilter
4     (mathematically identical to diagonal_recurrence, run once per mode
5     across all training signals at once)."""
6     return np.stack([lfilter([1.0], [1.0, -1], x_delayed, axis=-1) for li in lam],
7                     axis=0)

```

This computes *exactly* the same thing as `diagonal_recurrence`'s internal state trajectory  $m_i[k]$  for every mode  $i$  – but structured differently, for a different purpose. `scipy.signal.lfilter([1.0], [1.0, -1], ...)` runs the IIR recursion  $m[k] = \lambda_i m[k-1] + x[k]$  using a fast, compiled implementation, for *one* mode across *every* training signal at once (`x_delayed` can be a whole batch, shape (B, T), thanks to `axis=-1`). Doing this once per mode and stacking (`np.stack(...)`) gives a (p, B, T) array: every mode's complete feature trace, for every training signal, all at once – exactly the design matrix  $F$  that Part IV's regression needs. **This function is never used at deployment time** – it exists purely to build training features quickly; the deployed filter always uses `diagonal_recurrence`'s simple, one-sample-at-a-time loop.

## Part IV

# Cross-Validated Ridge Regression

```

1 def ridge_solve_c(features: np.ndarray, target: np.ndarray, reg: float) -> np.ndarray:
2     """c* = (F F^T + reg*diag(diag(F F^T)))^{-1} F t
3     -- per-mode-energy-scaled ridge, so modes of very different natural
4     magnitude (slow vs. fast decay) are regularized proportionally, not
5     uniformly."""
6     p = features.shape[0]
7     F = features.reshape(p, -1)
8     t = target.reshape(-1)
9     G = F @ F.T
10    reg_matrix = reg * np.diag(np.diag(G) + 1e-300)
11    return np.linalg.solve(G + reg_matrix, F @ t)

```

This is equation (5) of the theory document (§11, “Ridge regression: the fix, derived”), implemented directly. `features.reshape(p, -1)` flattens the (p, B, T) feature array down to (p, B\*T) – every mode's value at every (signal, timestep) pair becomes one long row, so the regression treats every training sample as one data point, regardless of which signal or timestep it came from.  $G = F @ F.T$  is the Gram matrix  $FF^T$  from Theorem 9.2. `np.diag(np.diag(G))` extracts just  $G$ 's diagonal (each mode's own energy  $\|f_i\|^2$ ) and rebuilds a diagonal matrix from it – this is  $D$  from that same equation, the per-mode-energy scaling that keeps the penalty proportional across modes of very different natural magnitude. `np.linalg.solve(G + reg_matrix, F @ t)` solves the resulting linear system directly (numerically preferable to explicitly computing a matrix inverse), returning  $c_{\text{ridge}}^*$ .

```

1 def cv_select_reg(features: np.ndarray, target: np.ndarray, n_signals: int,
2                 candidates, n_folds: int = 4):
3     """Honest k-fold cross-validation restricted to the TRAINING signals
4     only -- never touches the data the final filter will be judged on.
5     Returns the candidate reg with the lowest average held-out error."""

```

```

6 fold_size = n_signals // n_folds
7 idx = np.arange(n_signals)
8 folds = []
9 for f in range(n_folds):
10     val_idx = idx[f * fold_size:(f + 1) * fold_size]
11     fit_idx = np.setdiff1d(idx, val_idx)
12     folds.append((fit_idx, val_idx))
13
14 best_reg, best_cv = None, np.inf
15 for reg in candidates:
16     fold_rmses = []
17     for fit_idx, val_idx in folds:
18         c = ridge_solve_c(features[:, fit_idx, :], target[fit_idx], reg)
19         pred = np.tensordot(c, features[:, val_idx, :], axes=(0, 0))
20         err = pred - target[val_idx]
21         fold_rmses.append(float(np.sqrt(np.mean(err ** 2))))
22     cv = np.mean(fold_rmses)
23     if cv < best_cv:
24         best_cv, best_reg = cv, reg
25 return best_reg, best_cv

```

The first block partitions the  $n\_signals$  training signals into  $n\_folds$  disjoint groups and, for each group, records which *other* signals it should be fit on (`fit_idx`) versus which signals it should be validated against (`val_idx`, held out from that particular fit). The second block is the actual cross-validation: for every candidate regularization strength, fit `ridge_solve_c` on each fold's `fit_idx` signals only, evaluate the resulting  $c$  on that fold's held-out `val_idx` signals (which it never saw), and average the held-out error across all folds. Whichever candidate has the lowest average held-out error is returned. **Every number this function computes comes from the training signals alone** – the true test set is never referenced anywhere in this function, which is exactly the honesty property required of a regularization-strength choice (§12 of the theory document).

## Part V

# The Assembled Pipeline

```

1 class CompressedFractionalFilter:
2     """The complete method. Usage:
3         f = CompressedFractionalFilter(alpha=0.5, h=0.01, L=32, p=16)
4         f.fit(train_signals, j_max=10000)
5         y_hat = f.predict(x)
6     """
7
8     def __init__(self, alpha: float, h: float, L: int, p: int):
9         self.alpha, self.h, self.L, self.p = alpha, h, L, p
10        self.lam = None # decay rates: fixed forever once computed, never re-fit
11        self.c = None # linear weights: the only thing data ever touches
12        self.reg_used = None

```

The constructor just records the four design choices ( $\alpha$ , sample time  $h$ , local-window length  $L$ , number of tail modes  $p$ ) and initializes the two things that get filled in by fitting: `self.lam` (the decay rates – set once, in `fit`, and never touched again by anything) and `self.c` (the linear weights – the only quantity this whole class ever lets data influence).



```

1  def _design_decay_rates(self, j_max: int, w: np.ndarray):
2      """Step 1-2 of the algorithm: pure mathematics, no data."""
3      lam, c_analytical = soe_tail_kernel(self.alpha, self.L, self.p, w, j_max)
4      self.lam = lam
5      return c_analytical # kept only as a reference/fallback value

```

A thin wrapper around Part II's `soe_tail_kernel`, setting `self.lam`. Note that this function is called *before* any training signal is even generated inside `.fit` below – the decay rates literally cannot depend on the training data, because they are computed before that data exists in the function's scope.

```

1  def fit(self, train_signals, j_max: int, reg_candidates=None):
2      """train_signals: list/array of training signals (all same length).
3      Runs the complete pipeline: derive lambda analytically, build
4      features, cross-validate regularization, solve for c."""
5      if reg_candidates is None:
6          reg_candidates = [1e-11, 3e-11, 1e-10, 3e-10, 1e-9, 3e-9, 1e-8, 1e-7, 1e-6,
7                             1e-5]
8
9      n_signals = len(train_signals)
10     n_samples = len(train_signals[0])
11     w = gl_weights(self.alpha, max(j_max, n_samples) + 1)
12
13     gold = np.stack([full_gl_derivative(x, self.alpha, self.h, w) for x in
14                      train_signals])
15     local = np.stack([local_exact_term(x, self.alpha, self.h, self.L, w) for x in
16                      train_signals])
17     x_delayed = np.stack([delay(x, self.L) for x in train_signals])
18     target = gold - local # what the tail alone must explain
19
20     self._design_decay_rates(j_max, w) # step 1-2: fixes self.lam
21     # permanently
22     feats = mode_features(self.lam, x_delayed) # step 4: build feature matrix
23
24     best_reg, best_cv = cv_select_reg(feats, target, n_signals, reg_candidates) #
25     # step 5
26     self.c = ridge_solve_c(feats, target, best_reg) # step 6: final fit on all
27     # training data
28     self.reg_used = best_reg
29     return self

```

This is the whole algorithm from §13 of the theory document, assembled: compute enough exact GL weights to cover both the training signals and the intended quadrature design horizon (`max(j_max, n_samples)`); compute each training signal's true (gold-standard) derivative and its exact local term, so `target = gold - local` is precisely what the tail modes need to reproduce; derive the decay rates from pure mathematics (`_design_decay_rates`); build the training feature matrix (`mode_features`); cross-validate the regularization strength using training data only (`cv_select_reg`); and finally solve once, on *all* the training data (not just one fold), for the deployed `c`.

**A real bug, caught while writing this document.** An earlier version of `predict` (below) multiplied the tail output by an extra factor of  $h^{-\alpha}$ . This is *correct* for the raw SOE-quadrature `c` from Part II (which is fit directly against the unscaled kernel  $w_j$ ), but *wrong* for the ridge-fit `self.c` here – because `target` above is built from `gold` and



local, both of which *already* apply  $h^{-\alpha}$  internally (look back at `full_gl_derivative` and `local_exact_term` in Part I). Fitting `c` against an already-scaled target means `c` silently absorbs the scale factor into itself; applying  $h^{-\alpha}$  again at prediction time double-counted it, inflating errors by roughly  $10\times$  for this project's  $\alpha = 0.5, h = 0.01$  setup. Caught by comparing the class's output against an independent, by-hand recomputation of the same prediction – the same discipline used throughout this entire project's development, applied here to its own final, consolidated code before trusting it.

```

1 def predict(self, x: np.ndarray, w: np.ndarray = None) -> np.ndarray:
2     """Step 7: deploy. Identical cost to the pure classical filter --
3     L+p multiply-adds per sample."""
4     if w is None:
5         w = gl_weights(self.alpha, self.L)
6     local = local_exact_term(x, self.alpha, self.h, self.L, w)
7     x_delayed = delay(x, self.L)
8     tail = diagonal_recurrence(x_delayed, self.lam, self.c)
9     return local + tail

```

The deployed filter, matching step 7 of the algorithm exactly: compute the exact local term (cheap, exact,  $L$  multiply-adds), shift the input by  $L$  samples, run it through the fixed-decay-rate recursion from Part III using the fitted `self.c`, and add the two pieces together. No fitting, no cross-validation, no regression happens here – by the time `predict` runs, `self.lam` and `self.c` are just two fixed arrays of numbers, and this function is exactly as cheap to run as the pure classical filter from Part II, because structurally it *is* the same filter, just with better numbers loaded into `c`.

## 2 Putting it together: a complete usage example

```

1 from fracmem_final import CompressedFractionalFilter, gl_weights, full_gl_derivative
2
3 ALPHA, H, L, P = 0.5, 0.01, 32, 16
4
5 # a handful of representative training signals (any signals resembling
6 # what the filter will actually see in deployment)
7 train_signals = [ ... ]      # e.g. 8 arrays, each a typical input trace
8
9 f = CompressedFractionalFilter(alpha=ALPHA, h=H, L=L, p=P)
10 f.fit(train_signals, j_max=10000) # steps 1-6: pure math for lambda, CV+ridge for c
11
12 # deploy: identical cost to the pure classical filter, forever
13 y_hat = f.predict(new_signal)

```

That is the entire public interface: construct with the four design parameters, call `.fit` once (offline, using representative training signals), then call `.predict` as many times as needed at deployment time – each call costs exactly  $L + p$  multiply-adds, independent of how long `new_signal` is or how many times `.predict` has already been called.

## 3 Verified results, from this exact code

Running the code above with  $\alpha = 0.5$ ,  $h = 0.01$ ,  $L = 32$ ,  $p = 16$ , on the standard 8-signal training set and 16-signal held-out test set used throughout this project:

| Configuration                                                                        | Test RMSE               |
|--------------------------------------------------------------------------------------|-------------------------|
| Pure classical SOE (Part II alone, zero data; <code>soe_tail_kernel</code> directly) | $9.8234 \times 10^{-4}$ |
| <code>CompressedFractionalFilter</code> , single fit (training seed 0)               | $7.047 \times 10^{-5}$  |
| <code>CompressedFractionalFilter</code> , mean over 5 independent training draws     | $1.758 \times 10^{-4}$  |

Table 1: Reproduced directly from `fracmem_final.py` after the scale-convention fix above; matches the numbers reported throughout this project’s own development to within floating-point precision.